Document Intelligence Platform

Extract, validate and serve data from invoices and financial statements



AI Engineer Internship · Technical case study · <Your name>

The problem: no two documents look alike

- Invoices and statements come as native or scanned PDF, JPG or PNG
- Layouts, image quality and OCR errors differ for every document
- Numbers must reconcile (totals, balances, cash flows) before anyone trusts them
- Result must be reusable: REST API, dashboard, stored history

4 — **document types**
Invoice · Balance sheet · P&L · Cash flow

≤ 3 — **pages per file**
PDF, JPG or PNG; validated first

0 — **invented values**
Missing means null, never guessed

One pipeline, six controlled stages

1 Validate file

Type from content, size, corruption, page limit

2 Independent OCR

PDF text layer or Tesseract

3 LLM extraction

Gemini reads the file into a schema

4 Financial checks

Deterministic PASS / FAIL / N/A

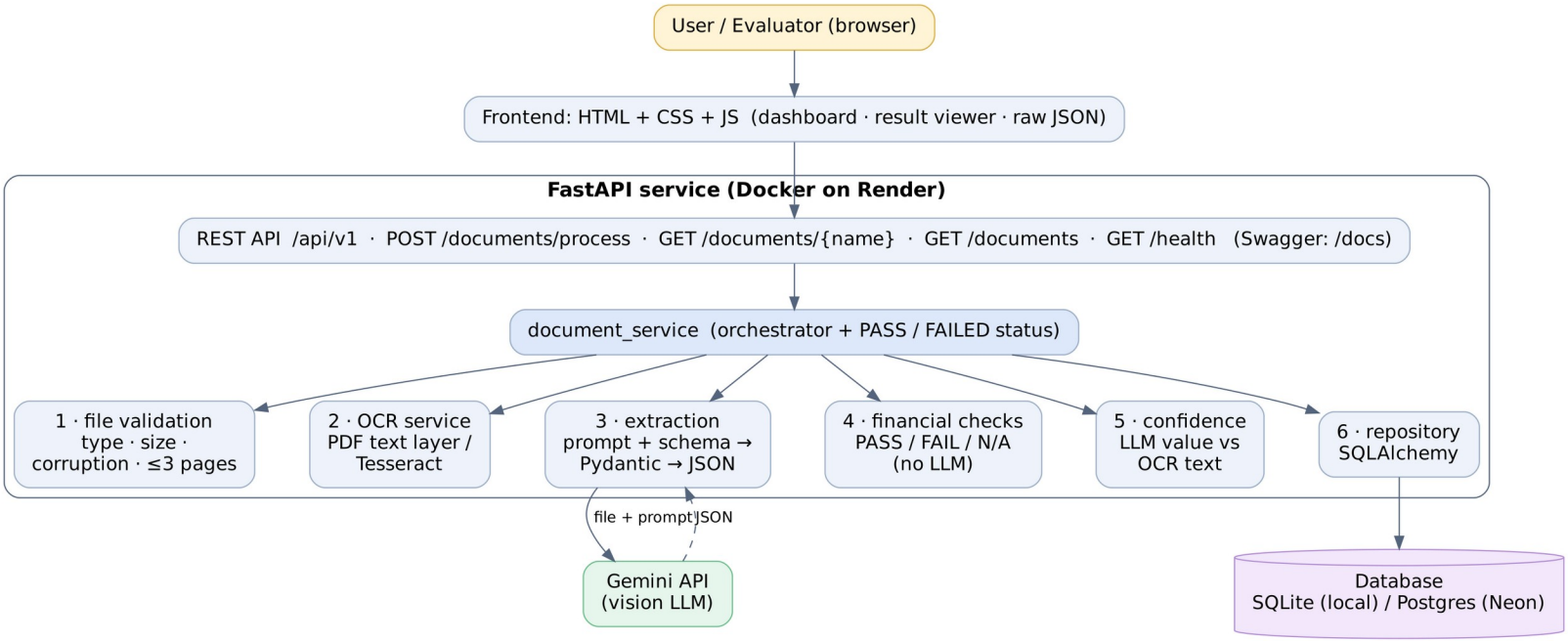
5 Evidence + confidence

Source text, page, OCR agreement

6 Store + serve

Database, REST API, dashboard

Architecture: thin API, separated services



The LLM never decides PASS/FAIL: validation is plain Python on the extracted numbers.

Extraction: schema-bound and evidence-first

- Vision LLM reads the PDF/image directly, temperature 0
- Per-type prompt + JSON schema; output parsed by Pydantic
- Invalid JSON is re-asked once, then fails cleanly
- Every value carries source_text and page_number
- Unreadable or absent values stay null

Statement row model

```
label: "Total"
row_type: item|subtotal|total|heading
section: assets|capital_and_liabilities
canonical_key: total_income
values: { "FY2023": 204666.10, "FY2022": 167695.40 }
source_text, page_number
```

Financial validation: every check shows its working

Document	Checks (per period / year)
Invoice	qty × price = line · Σ lines vs subtotal · subtotal + tax − discount = total (tax-inclusive handled) · cash − total = change
Balance sheet	assets = capital & liabilities · Σ asset items · Σ liability items
Profit & loss	income · expenditure · profit before minority · profit to group · appropriation
Cash flow	op + inv + fin (+ FX) = net change · opening + net change = closing

Each check returns: formula · operands · calculated · reported · variance · tolerance · PASS / FAIL / NOT_APPLICABLE

Confidence you can explain

```
confidence = 0.60 + 0.35 × share of value found in OCR text
```

0.95

Corroborated

The value appears in the OCR text

0.60

Not corroborated

Flagged low (< 0.80) and highlighted

none

Null or no OCR

No score shown; never faked

Limit: OCR can miss stamps and handwriting, so low means not corroborated, not wrong.

API, dashboard and persistence

REST API

- POST /api/v1/documents/process
- GET /api/v1/documents/{name}
- GET /api/v1/documents
- GET /api/v1/health
- One error shape, correct status codes, no stack traces

Dashboard

- Pick type, upload, process
- Searchable, filterable list
- Result view: fields, tables, evidence
- Missing, low-confidence, failed checks highlighted
- Raw JSON view, copy and download

Persistence

- SQLAlchemy repository
- SQLite locally, Postgres in production
- History kept; GET returns the latest
- Failed attempts stored too
- Files processed in memory, not saved

Testing: model faked, everything else real

40

automated tests passing

pytest · clean-venv install verified

- File validation: valid, empty, renamed, corrupt, 4-page PDF
- Financial maths: pass, fail, NOT_APPLICABLE, per-year, tax-inclusive receipt
- Extraction: normalisation, retry on bad JSON, null stays null
- API flow: process, latest-by-name, list, 415 / 400 / 422 / 404 / 502 / 500
- Real scanned PDF and receipt image through OCR confidence

Deployment, limitations and next steps

Deployment

- Docker image with Tesseract
- Render web service, health-checked
- Neon Postgres for durable data
- Secrets only in environment variables

Known limits

- Handwriting and stamps can be misread
- Synchronous, 5 to 40 s per file
- Free-tier rate limits and cold starts
- No authentication yet

Production next

- Async queue with status polling
- Evaluation set for field-level accuracy
- Original-file viewer with evidence boxes
- Alembic, metrics, rate limiting